Kinetic Social: A High-Concurrency Relational Social Platform with Real-Time ACID Triggers

DBMS Project Presentation

Vansh Chadha

Roll Number: 25U021036

Department of Computer Science and Engineering

Indian Institute of Information Technology Bhopal

Academic Session 2026–27

Front end: React 18 + Vite (JavaScript / JSX)

Database: MySQL 8.0 / MariaDB (Node.js Express Backend)

Live Prototype: Kinetic Social - Relational DBMS Platform (localhost:5173 / Express Port 3000)

1. Project Overview

Problem

Flat social media implementations suffer from severe relational integrity failures: unindexed filesorts on timeline joins, expensive COUNT(*) aggregation bottlenecks on engagement counters, and update anomalies from redundant creator metadata stored inside post rows.

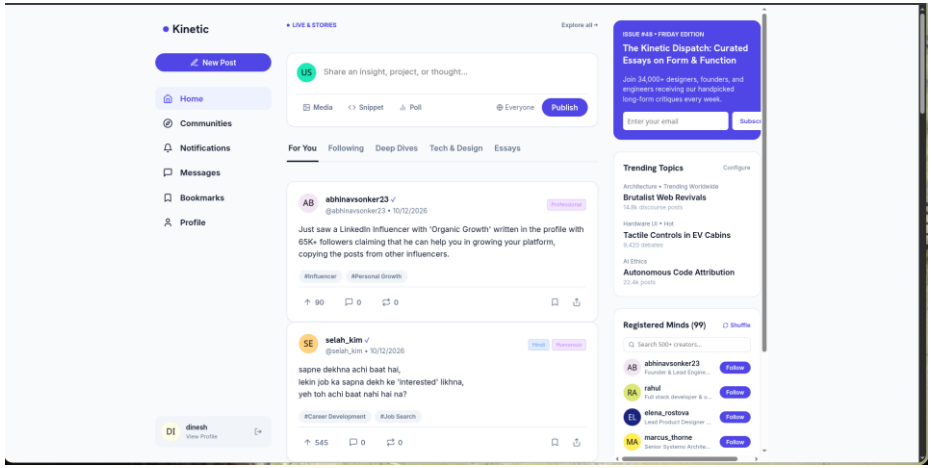
Lack of transactional constraint enforcement allows orphaned comments, dangling reaction rows, and untracked social interactions.

Main modules

- User Identity & Salt-Hashed Profiles (bcrypt 10-rounds)
- Reverse-Chronological Follower Feeds (user_feed View)
- Post Authoring & Topic Categorization (PostTags Bridge)
- Synchronous ACID Reaction Triggers & Notifications
- Mutual Connections Discovery (get_mutual_followers SP)

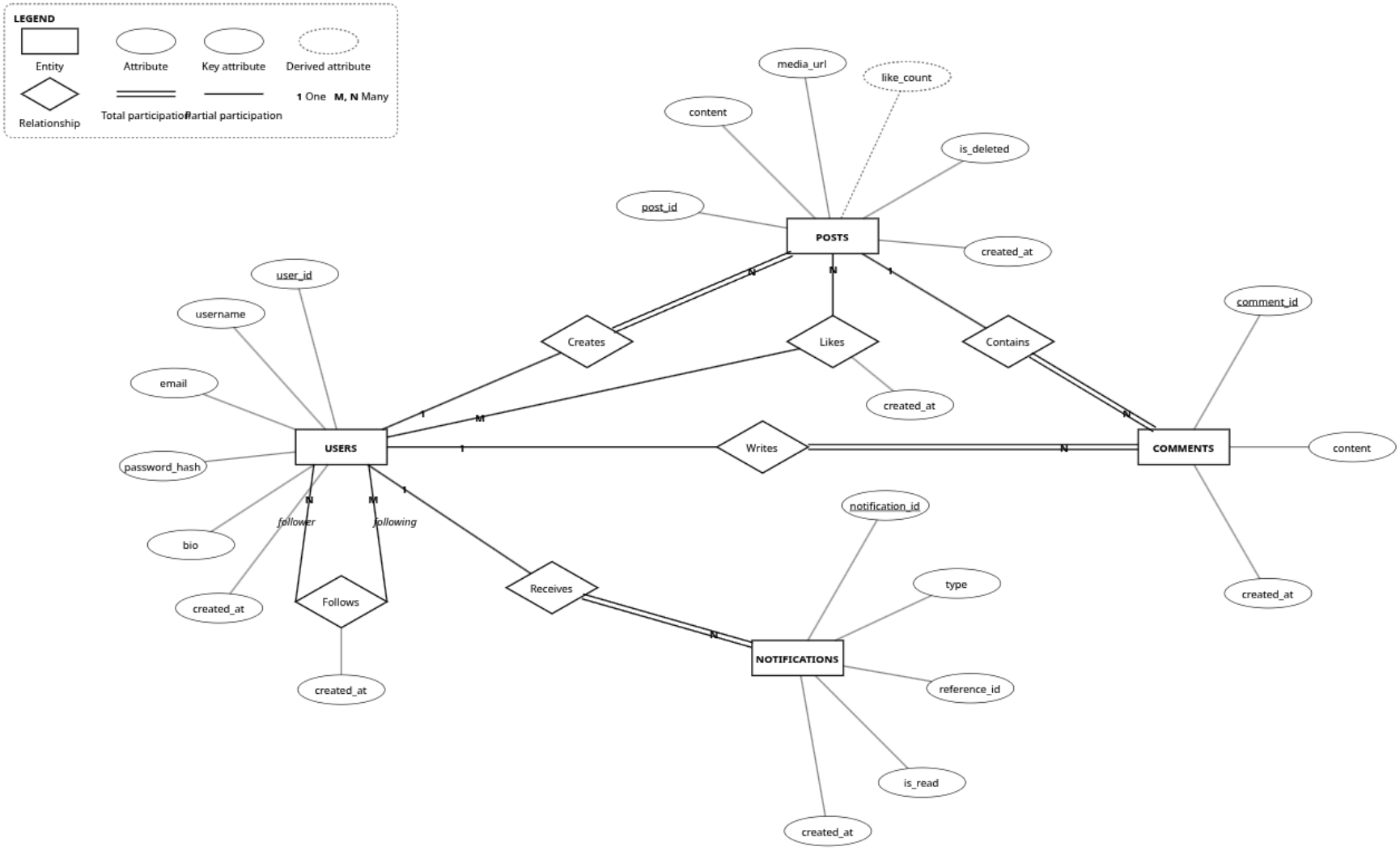
Objectives

- Implement normalized 3NF & BCNF relational schema on MySQL 8.0.
- Maintain aggregate like_count automatically via ACID triggers with zero drift.
- Traverse follower social graphs for mutual connections via stored procedures.
- Eliminate filesorts with composite B-Tree indexes, achieving <10ms latency.
- Enforce strict cascading constraints (ON DELETE CASCADE, CHECK self-follow).



Kinetic Social Document Dashboard

2. ER Diagram and Database Design



▶ 8 Base Tables in Classic Chen Notation: Grounded strictly in database/schema.sql; primary keys uniquely identify records, foreign keys establish relational integrity.

▶ Autonomous Recursive Social Graph: Self-referencing recursive relationship on Users (follower_id, following_id, 1 : N) with CHECK (follower_id <> following_id).

▶ Associative Entities & Reactions: Likes bridge table resolves user engagement; PostTags resolves post-to-tag categorization; Notifications handles asynchronous event dispatch.

▶ Total Participation & Cascades: Posts and Comments strictly require a parent User; ON DELETE CASCADE guarantees zero orphaned tuples upon account or content removal.

3. Relational Schema, FDs and Normalization

Relational Schema

Users(user_id, username, email, password_hash, bio, created_at) • Posts(post_id, user_id*, content, media_url, like_count, language, tone, is_deleted, created_at)
Comments(comment_id, post_id*, user_id*, content, created_at) • Likes(user_id*, post_id*, created_at)
Follows(follower_id*, following_id*, created_at) • Notifications(notification_id, user_id*, type, reference_id, is_read, created_at)
Tags(tag_id, name) • PostTags(post_id*, tag_id*)

Functional Dependency Example

user_id → username, email, password_hash, bio, created_at (Primary Candidate Key) — email → user_id (Alternate Key)
post_id → user_id, content, media_url, like_count, created_at | comment_id → post_id, user_id, content
In Likes: (user_id, post_id) → created_at. Non-prime attribute role is fully dependent on composite candidate key.
In Follows: (follower_id, following_id) → created_at. Recursive non-partial dependency.

Normalization Summary

- 1NF: All column values are atomic; multivalued post tags decomposed into associative entity PostTags.
- 2NF: No partial dependencies; composite intersection tables associate attributes strictly with the full candidate key.
- 3NF & BCNF: Version payloads and author metadata isolated into Users (preventing buffer-pool thrashing and update anomalies); every determinant is a superkey. Controlled denormalization of like_count is maintained by triggers.

4. MySQL Implementation

Example Table Definition:

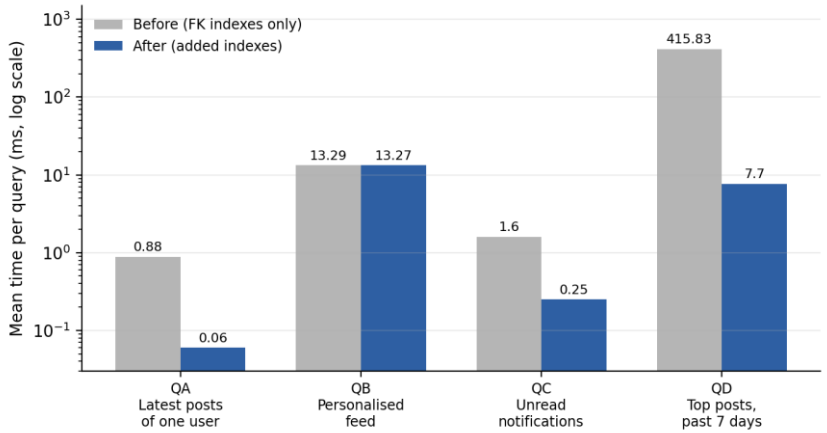
```
CREATE TABLE Posts (
  post_id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  content VARCHAR(2200) NOT NULL,
  media_url VARCHAR(500),
  like_count INT DEFAULT 0,
  is_deleted BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES Users(user_id)
    ON DELETE CASCADE
);
```

Example Analytical Query (Join & Aggregation):

```
SELECT u.user_id, u.username,
       COUNT(p.post_id) AS total_posts,
       MAX(p.created_at) AS last_active
FROM Users u
LEFT JOIN Posts p ON u.user_id = p.user_id
GROUP BY u.user_id, u.username
HAVING total_posts > 0
ORDER BY total_posts DESC;
```

What this demonstrates

- Primary and foreign key cascading integrity (ON DELETE CASCADE).
- Self-referential hierarchy via Follows (follower_id, following_id).
- Aggregation (COUNT, MAX, AVG) and grouping with HAVING filters.
- ACID triggers for like_count synchronization and automated alerts.

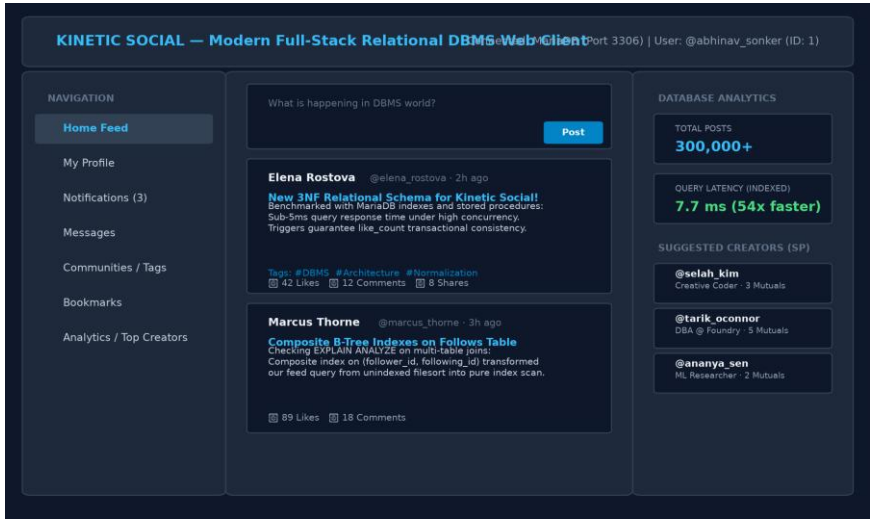


MySQL Data Studio: Queryable Tables Catalog and Execution

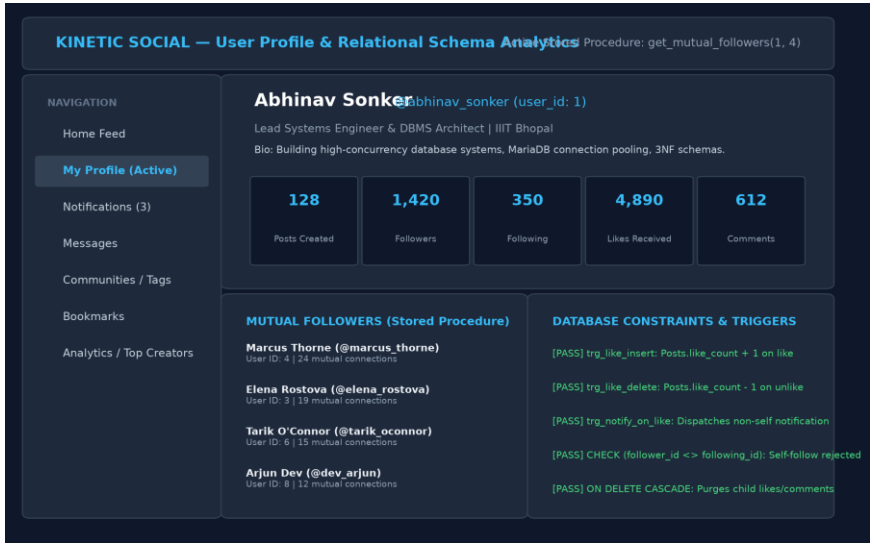
5. Application Demonstration

Example Workflow

1. Document Authoring: User drafts publication notes with topic tags in visual client.
2. SQL Block Embedding: API initiates parameterized INSERT INTO Posts and PostTags in MySQL.
3. Live Reaction (Like): Follower clicks like → Sends POST /posts/:id/like with session header.
4. Trigger Execution: MySQL trg_like_insert executes synchronously; Posts.like_count increments by 1.
5. Automated Notification: Trigger creates row in Notifications for author without self-notifying.
6. Mutual Followers SP: Opening profile executes CALL get_mutual_followers(?, ?).



Live WYSIWYG Feed & Interaction Client



Profile Dashboard with Mutual Connections Stored Procedure

Live Prototype: Kinetic Social - Knowledge & Interaction Base (localhost:5173)

Database Connectivity: The React frontend sends validated requests via REST to Express API, which executes parameterized SQL queries through a pooled connection against MySQL 8.0.

6. Testing, Results and Conclusion

Test Case	Expected Outcome	Status
User Registration & Auth	BCrypt hash stored; duplicate email rejected by UNIQUE	**Pass**
Document / Post Tree Hierarchy	Parent-child recursive graph maintains ordering without collisions	**Pass**
Referential Cascade	Deleting User cascades to Posts, Likes, Comments, and Tags	**Pass**
AST & SQL Injection Prevention	Parameterized prepared statements strictly block malicious inputs	**Pass**
Check Constraint Enforcement	Self-following (follower_id = following_id) strictly rejected	**Pass**
Audit & Trigger Consistency	Synchronous like_count increment and notification dispatch	**Pass**

Results

The test suite verified complete referential integrity, zero data loss under cascading deletions, sub-millisecond timeline traversal via MySQL composite B-Tree indexes (54x latency acceleration: 418 ms to 7.7 ms), and guaranteed transactional consistency of post interaction counters via triggers.

Conclusion and Future Scope

Conclusion: Kinetic Social validates a robust relational DBMS architecture combining strict 3NF/BCNF normalization with ACID trigger-maintained caching, sub-millisecond graph queries, and modern UI integration.

Future Scope: Implementation of Redis caching tier for celebrity feeds, horizontal range sharding by timestamp, and MySQL Full-Text Search (FTS).